3. spark.sql

Overview

- `Spark SQL` is a library included in `Spark` since version 1.3
- It provides an **easier interface to process tabular data**
- Instead of `RDD`s, we deal with `DataFrame`s
- Since `Spark` 1.6, there is also the concept of `Dataset`s, but only for `Scala` and `Java`

`SparkContext` and `SparkSession`

- Before `Spark 2`, there was only `SparkContext` and `SQLContext`
- All core functionality was accessed with `SparkContext`
- All `SQL` functionality needed the `SQLContext`, which can be created from an `SparkContext`
- Since `Spark 2`, there is now the `SparkSession` class
- `SparkSession` is now the **global entry-point** for everything `Spark`-related

SparkContext and SparkSession

Before Spark 2

```
>>> from pyspark import SparkConf, SparkContext
>>> from pyspark.sql import SQLContext

>>> conf = SparkConf().setAppName(appName).setMaster(master)
>>> sc = SparkContext(conf = conf)
>>> sql_context = new SQLContext(sc)
```

Since Spark 2

```
>>> from pyspark.sql import SparkSession

>>> spark = SparkSession \
        .builder \
        .appName(appName) \
        .master(master) \
        .config("spark.some.config.option", "some-value") \
        .getOrCreate()
```

3.1. DataFrame

`DataFrame`

- The main entity of `Spark SQL` is the `DataFrame`
- A DataFrame is actually an `RDD` of `Row`s with a **schema**
- A schema gives the **names of the columns** and their **types**
- `Row` is a class representing a row of the `DataFrame`.
- It can be used almost as a `python list`, with its size equal to the number of columns in the schema.

DataFrame

```
>>> from pyspark.sql import Row

>>> row1 = Row(name="John", age=21)
>>> row2 = Row(name="James", age=32)
>>> row3 = Row(name="Jane", age=18)
>>> row1['name']
'John'
```

```
>>> df = spark.createDataFrame([row1, row2, row3])
>>> df
DataFrame[age: bigint, name: string]
```

```
>>> df.show()
+-----+---+
| name|age|
+-----+---+
| John| 21|
|James| 32|
| Jane| 18|
+-----+---+
```

DataFrame

```
>>> df.printSchema()
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)
```

You can ccess to the underlying **RDD** object using **.rdd**

```
>>> print(df.rdd.toDebugString().decode("utf-8"))
(6) MapPartitionsRDD[16] at javaToPython at NativeMethodAccessorImpl.java:(
 |  MapPartitionsRDD[15] at javaToPython at NativeMethodAccessorImpl.java:(
 |  MapPartitionsRDD[14] at javaToPython at NativeMethodAccessorImpl.java:(
 |  MapPartitionsRDD[13] at applySchemaToPythonRDD at NativeMethodAccessorI
 |  MapPartitionsRDD[12] at map at SerDeUtil.scala:137 []
 |  MapPartitionsRDD[11] at mapPartitions at SerDeUtil.scala:184 []
 |  PythonRDD[10] at RDD at PythonRDD.scala:53 []
 |  ParallelCollectionRDD[9] at parallelize at PythonRDD.scala:195 []
```

```
>>> df.rdd.getNumPartitions()
6
```

Creating DataFrames

- We can use the method `createDataFrame` from the SparkSession instance
- Can be used to create a `Spark DataFrame` from:
  - a `pandas.DataFrame` object
  - a local python list
  - an RDD
- Full documentation can be found in the `[API docs]`

Creating DataFrames

```
>>> rows = [
        Row(name="John", age=21, gender="male"),
        Row(name="James", age=25, gender="female"),
        Row(name="Albert", age=46, gender="male")
    ]
>>> df = spark.createDataFrame(rows)
>>> df.show()
+---+------+------+
|age|gender|  name|
+---+------+------+
| 21|  male|  John|
| 25|female| James|
| 46|  male|Albert|
+---+------+------+
```

Creating DataFrames

```
>>> column_names = ["name", "age", "gender"]
>>> rows = [
        ["John", 21, "male"],
        ["James", 25, "female"],
        ["Albert", 46, "male"]
    ]
>>> df = spark.createDataFrame(rows, column_names)
>>> df.show()
+------+---+------+
|  name|age|gender|
+------+---+------+
|  John| 21|  male|
| James| 25|female|
|Albert| 46|  male|
+------+---+------+
```

Creating DataFrames

```
>>> column_names = ["name", "age", "gender"]
>>> rdd = sc.parallelize([
        ("John", 21, "male"),
        ("James", 25, "female"),
        ("Albert", 46, "male")
    ])
>>> df = spark.createDataFrame(rdd, column_names)
>>> df.show()
+------+---+------+
|  name|age|gender|
+------+---+------+
|  John| 21|  male|
| James| 25|female|
|Albert| 46|  male|
+------+---+------+
```

3.2. Schemas and types

Schema and Types

- A `DataFrame` always contains a **schema**
- The schema defines the **column names** and **types**
- In all previous examples, the schema was **inferred**
- The schema of a `DataFrame` is represented by the class `types.StructType` [API doc]
- When creating a `DataFrame`, the schema can be either **inferred** or **defined by the user**

```
>>> df.schema
StructType(List(StructField(name,StringType,true),StructField(age,IntegerTy
```

Creating a custom Schema

```
>>> from pyspark.sql.types import *

>>> schema = StructType([
        StructField("name", StringType(), True),
        StructField("age", IntegerType(), True),
        StructField("gender", StringType(), True)
    ])
>>> rows = [("John", 21, "male")]
>>> df = spark.createDataFrame(rows, schema)
>>> df.printSchema()
>>> df.show()
root
|-- name: string (nullable = true)
|-- age: integer (nullable = true)
|-- gender: string (nullable = true)

+----+---+------+
|name|age|gender|
+----+---+------+
|John| 21|  male|
+----+---+------+
```

Types supported by `Spark SQL`

- `StringType`
- `IntegerType`
- `LongType`
- `FloatType`
- `DoubleType`
- `BooleanType`
- `DateType`
- `TimestampType`
- `...`
- The full list of types can be found in `[API doc]`

3.3. Reading data

Reading data from sources

- Data is usually read from **external sources**
  (move the **algorithms**, not the **data**)

- `Spark SQL` provides **connectors** to read from many different sources:

  - Text files (`CSV`, `JSON`)
  - Distributed tabular files (`Parquet`, `ORC`)
  - In-memory data sources (`Apache Arrow`)
  - General relational Databases (via `JDBC`)
  - Third-party connectors to connect to many other databases
  - And you can create your own connector for `Spark` (in `Scala`)

Reading data from sources

- In all cases, the syntax is similar: `spark.read.{source}(path)`
- Spark supports different **file systems** to look for the data:
  - Local files: "`file://path/to/file`" or just "`path/to/file`"
  - `HDFS` (Hadoop filesystem): "`hdfs://path/to/file`"
  - `Amazon S3`: "`s3://path/to/file`"

Reading from a CSV file

```
>>> df = spark.read.csv("/path/to/file.csv")
```

```
>>> path = "/path/to/file.csv"
>>> df = spark.read.option("header", "true").csv(path)
```

```
>>> df = (spark.read
          .format('csv')
          .option('header', 'true')
          .option('sep', ";")
          .load("/path/to/file.csv")
     )
```

```
>>> df = spark.read.csv(path, sep=";", header=True)
```

Reading from a CSV file
Main Options

Some important options of the `CSV` reader are listed here:

option	description
`sep`	The separator character
`header`	If "true", the first line contains the column names
`inferSchema`	If "true", the column types will be guessed from the contents
`dateFormat`	A string representing the format of the date columns

The full list of options can be found in the `API Docs`

Reading from other file types

```
# JSON file
>>> df = spark.read.json("/path/to/file.json")
>>> df = spark.read.format("json").load("/path/to/file.json")
```

```
# Parquet file (distributed tabular data)
>>> df = spark.read.parquet("hdfs://path/to/file.parquet")
>>> df = spark.read.format("parquet").load("hdfs://path/to/file.parquet")
```

```
# ORC file (distributed tabular data)
>>> df = spark.read.orc("hdfs://path/to/file.orc")
>>> df = spark.read.format("orc").load("hdfs://path/to/file.orc")
```

Reading from external databases

- We can use `JDBC` drivers (Java) to read from relational databases
- Examples of databases: `Oracle`, `PostgreSQL`, `MySQL`, etc.
- The `java` driver file must be uploaded to the cluster before trying to access
- This operation can be **very heavy**. When available, specific connectors should be used
- Specific connectors are often provided by **third-party libraries**

Reading from external databases

```
>>> df = spark.read.format("jdbc") \
        .option("url", "jdbc:postgresql:dbserver") \
        .option("dbtable", "schema.tablename") \
        .option("user", "username") \
        .option("password", "p4ssw0rd") \
        .load()
```

or

```
>>> df = spark.read.jdbc(
        url="jdbc:postgresql:dbserver",
        table="schema.tablename"
        properties={
            "user": "username",
            "password": "p4ssw0rd"
        }
    )
```

3.4. Queries in `spark.sql`

Performing queries

- `Spark SQL` was created to be compatible with SQL queries
- So it supports actual `SQL` queries to be performed on `DataFrame`s
- First, the `DataFrame` must be **tagged as a temporary view**
- Then, the queries can be applied using `spark.sql`

Performing queries

```
>>> column_names = ["name", "age", "gender"]
>>> rows = [
        ["John", 21, "male"],
        ["Jane", 25, "female"]
    ]
>>> df = spark.createDataFrame(rows, column_names)

# Create a temporary view from the DataFrame
>>> df.createOrReplaceTempView("new_view")

# Apply the query
>>> query = "SELECT name, age FROM new_view WHERE gender='male'"
>>> men_df = spark.sql(query)
>>> men_df.show()
+----+---+
|name|age|
+----+---+
|John| 21|
+----+---+
```

Using the API

- Although allowing `SQL` queries is a very powerful feature, it's **not the best way** to code a complex logic
- Errors are **harder to find** in strings
- Using queries makes the code **less modularizable**
- So `Spark SQL` provides a **full API** with **SQL-like operations**
- It's **the best way** to code complex logic when using `Spark SQL`

Basic Operations

operation	description
`select`	Chooses columns from the table
`where`	Filters rows based on a boolean rule
`limit`	Limits the number of rows
`orderBy`	Sorts the DataFrame based on one or more columns
`alias`	Changes the name of a column
`cast`	Changes the type of a column
`withColumn`	Adds a new column

SELECT

```
# In a SQL query:
>>> query = "SELECT name, age FROM table"

# Using Spark SQL API:
>>> df.select("name", "age").show()
+-----+---+
| name|age|
+-----+---+
| John| 21|
| Jane| 25|
+-----+---+
```

WHERE

```
# In a SQL query:
>>> query = "SELECT * FROM table WHERE age > 21"

# Using Spark SQL API:
>>> df.where("age > 21").show()

# Alternatively:
>>> df.where(df['age'] > 21).show()
>>> df.where(df.age > 21).show()
>>> df.select("*").where("age > 21").show()
+----+---+------+
|name|age|gender|
+----+---+------+
|Jane| 25|female|
+----+---+------+
```

LIMIT

```
# In a SQL query:
>>> query = "SELECT * FROM table LIMIT 1"

# Using Spark SQL API:
>>> df.limit(1).show()

# Or even
>>> df.select("*").limit(1).show()
+----+---+------+
|name|age|gender|
+----+---+------+
|Jane| 25|female|
+----+---+------+

# Note: The result is not deterministic!
```

ORDER BY

```
# In a SQL query:
>>> query = "SELECT * FROM table ORDER BY name ASC"

# Using Spark SQL API:
>>> df.orderBy(df.name.asc()).show()
+----+---+------+
|name|age|gender|
+----+---+------+
|Jane| 25|female|
|John| 21|  male|
+----+---+------+
```

ALIAS (name change)

```
# In a SQL query:
>>> query = "SELECT name, age, gender AS sex FROM table"

# Using Spark SQL API:
>>> df.select(df.name, df.age, df.gender.alias('sex')).show()
+----+---+------+
|name|age|   sex|
+----+---+------+
|John| 21|  male|
|Jane| 25|female|
+----+---+------+
```

CAST (type change)

```
# In a SQL query:
>>> query = "SELECT name, cast(age AS float) AS age_f FROM table"

# Using Spark SQL API:
>>> df.select(df.name, df.age.cast("float").alias("age_f")).show()

# Or
>>> new_age_col = df.age.cast("float").alias("age_f")
>>> df.select(df.name, new_age_col).show()
+----+-----+
|name|age_f|
+----+-----+
|John| 21.0|
|Jane| 25.0|
+----+-----+
```

Adding new columns

```
# In a SQL query:
>>> query = "SELECT *, 12*age AS age_months FROM table"

# Using Spark SQL API:
>>> df.withColumn("age_months", df.age * 12).show()

# Or
>>> df.select("*", (df.age * 12).alias("age_months")).show()
+----+---+------+----------+
|name|age|gender|(age * 12)|
+----+---+------+----------+
|John| 21|  male|       252|
|Jane| 25|female|       300|
+----+---+------+----------+

# Note: Using withColumn is preferable
```

Basic operations

- The **full list of operations** that can be applied to a `DataFrame` can be found in the `[DataFrame doc]`
- The **list of operations on columns** can be found in the `[Column docs]`

3.5. Column functions

Column functions

- Oftentimes, we need to make many **transformations using one or more functions**

- `Spark SQL` has a package called `functions` with many functions available for that

- Some of those functions are only for **aggregations**
  Examples: `avg`, `sum`, etc. We will cover them later

- Some others are for **column transformation** or **operations**
  Examples: `substr`, `concat`, `datediff`, `floor`, etc.

- The full list is, once again, in the `[API docs]`

Column functions

To use these functions, we first need to import them:

```
>>> from pyspark.sql import functions as fn
```

Note: the "`as fn`" part is important to **avoid confusion** with native `Python` functions such as "sum"

Numeric functions examples

```
>>> from pyspark.sql import functions as fn

>>> columns = ["brand", "cost"]
>>> df = spark.createDataFrame([
        ("garnier", 3.49),
        ("elseve", 2.71)
        ], columns)

>>> round_cost = fn.round(df.cost, 1)
>>> floor_cost = fn.floor(df.cost)
>>> ceil_cost = fn.ceil(df.cost)

>>> df.withColumn('round', round_cost)\
        .withColumn('floor', floor_cost)\
        .withColumn('ceil', ceil_cost)\
        .show()
+-------+----+-----+-----+----+
|  brand|cost|round|floor|ceil|
+-------+----+-----+-----+----+
|garnier|3.49|  3.5|    3|   4|
| elseve|2.71|  2.7|    2|   3|
+-------+----+-----+-----+----+
```

String functions examples

```
>>> from pyspark.sql import functions as fn

>>> columns = ["first_name", "last_name"]

>>> df = spark.createDataFrame([
        ("John", "Doe"),
        ("Mary", "Jane")
        ], columns)

>>> last_name_initial = fn.substring(df.last_name, 0, 1)
>>> name = fn.concat_ws(" ", df.first_name, last_name_initial)
>>> df.withColumn("name", name).show()
+----------+---------+------+
|first_name|last_name|  name|
+----------+---------+------+
|      John|      Doe|John D|
|      Mary|     Jane|Mary J|
+----------+---------+------+
```

Date functions examples

```
>>> from datetime import date
>>> from pyspark.sql import functions as fn

>>> df = spark.createDataFrame([
        (date(2015, 1, 1), date(2015, 1, 15)),
        (date(2015, 2, 21), date(2015, 3, 8)),
        ], ["start_date", "end_date"]
    )

>>> days_between = fn.datediff(df.end_date, df.start_date)
>>> start_month = fn.month(df.start_date)

>>> df.withColumn('days_between', days_between)\
        .withColumn('start_month', start_month)\
        .show()
+----------+----------+------------+-----------+
|start_date|  end_date|days_between|start_month|
+----------+----------+------------+-----------+
|2015-01-01|2015-01-15|          14|          1|
|2015-02-21|2015-03-08|          15|          2|
+----------+----------+------------+-----------+
```

Conditional transformations

- In the `functions` package is a **special function** called `when`
- This function is used to **create a new column** which value **depends on the value of other columns**
- `otherwise` is used to match "the rest"
- Combination between conditions can be done using `"&"` for "and" and `"|"` for "or"

Examples

```
>>> df = spark.createDataFrame([
        ("John", 21, "male"),
        ("Jane", 25, "female"),
        ("Albert", 46, "male"),
        ("Brad", 49, "super-hero")
    ], ["name", "age", "gender"])

>>> supervisor = fn.when(df.gender == 'male', 'Mr. Smith')\
        .when(df.gender == 'female', 'Miss Jones')\
        .otherwise('NA')

>>> df.withColumn("supervisor", supervisor).show()
+------+---+----------+----------+
|  name|age|    gender|supervisor|
+------+---+----------+----------+
|  John| 21|      male| Mr. Smith|
|  Jane| 25|    female|Miss Jones|
|Albert| 46|      male| Mr. Smith|
|  Brad| 49|super-hero|        NA|
+------+---+----------+----------+
```

User-defined functions

- When you need a **transformation** that is **not available** in the `functions` package, you can create a **User Defined Function** (UDF)
- **Warning**: the performance of this can be **very very low**
- So, it should be used **only** when you are **sure** the operation **cannot be done** with available functions
- To create an UDF, use `functions.udf`, passing a lambda or a named functions
- It is similar to the `map` operation of RDDs

Example

```
>>> from pyspark.sql import functions as fn
>>> from pyspark.sql.types import StringType

>>> df = spark.createDataFrame([(1, 3), (4, 2)], ["first", "second"])

>>> def my_func(col_1, col_2):
        if (col_1 > col_2):
            return "{} is bigger than {}".format(col_1, col_2)
        else:
            return "{} is bigger than {}".format(col_2, col_1)

>>> my_udf = fn.udf(my_func, StringType())

>>> df.withColumn("udf", my_udf(df['first'], df['second'])).show()
+-----+------+------------------+
|first|second|               udf|
+-----+------+------------------+
|    1|     3|3 is bigger than 1|
|    4|     2|4 is bigger than 2|
+-----+------+------------------+
```

3.6. Joins

Performing joins

- `Spark SQL` supports **joins** between two `DataFrame`
- As in normal `SQL`, a join **rule** must be defined
- The **rule** can either be a set of **join keys**, or a **conditional rule**
- Join with conditional rules in `Spark` can be **very heavy**
- **Several types of joins** are available, default is `"inner"`

Syntax is simple:

```
>>> left_df.join(other=right_df, on=cols, how=join_type)
```

- `cols` contains a column name or a list of column names
- `join_type` is the type of join

Examples

```
>>> from datetime import date

>>> products = spark.createDataFrame([
        ('1', 'mouse', 'microsoft', 39.99),
        ('2', 'keyboard', 'logitech', 59.99),
    ], ['prod_id', 'prod_cat', 'prod_brand', 'prod_value'])

>>> purchases = spark.createDataFrame([
        (date(2017, 11, 1), 2, '1'),
        (date(2017, 11, 2), 1, '1'),
        (date(2017, 11, 5), 1, '2'),
    ], ['date', 'quantity', 'prod_id'])

>>> # The default join type is the "INNER" join
>>> purchases.join(products, 'prod_id').show()
+-------+----------+--------+--------+----------+----------+
|prod_id|      date|quantity|prod_cat|prod_brand|prod_value|
+-------+----------+--------+--------+----------+----------+
|      1|2017-11-01|       2|   mouse| microsoft|     39.99|
|      1|2017-11-02|       1|   mouse| microsoft|     39.99|
|      2|2017-11-05|       1|keyboard|  logitech|     59.99|
+-------+----------+--------+--------+----------+----------+
```

Examples

```
>>> # We can also use a query string (not recommended)
>>> products.createOrReplaceTempView("products")
>>> purchases.createOrReplaceTempView("purchases")

>>> query = """SELECT * FROM
    (purchases AS prc INNER JOIN products AS prd
    on prc.prod_id = prd.prod_id)
    """


>>> spark.sql(query).show()
+----------+--------+-------+-------+--------+----------+----------+
|      date|quantity|prod_id|prod_id|prod_cat|prod_brand|prod_value|
+----------+--------+-------+-------+--------+----------+----------+
|2017-11-01|       2|      1|      1|   mouse| microsoft|     39.99|
|2017-11-02|       1|      1|      1|   mouse| microsoft|     39.99|
|2017-11-05|       1|      2|      2|keyboard|  logitech|     59.99|
+----------+--------+-------+-------+--------+----------+----------+
```

Examples

```
>>> new_purchases = spark.createDataFrame([
        (date(2017, 11, 1), 2, '1'),
        (date(2017, 11, 2), 1, '3'),
    ], ['date', 'quantity', 'prod_id_x'])

>>> join_rule = new_purchases.prod_id_x == products.prod_id

>>> new_purchases.join(products, join_rule, 'left').show()
+----------+--------+---------+-------+--------+----------+----------+
|      date|quantity|prod_id_x|prod_id|prod_cat|prod_brand|prod_value|
+----------+--------+---------+-------+--------+----------+----------+
|2017-11-02|       1|        3|   null|    null|      null|      null|
|2017-11-01|       2|        1|      1|   mouse| microsoft|     39.99|
+----------+--------+---------+-------+--------+----------+----------+
```

Performing joins: some remarks

- Spark **removes the duplicated column** in the `DataFrame` it outputs after a join operation
- When joining using columns **with nulls**, `Spark` just skips those

```
>>> df1.show()                  >>> df2.show()
+----+-----+                    +----+-----+
|  id| name|                    |  id| dept|
+----+-----+                    +----+-----+
| 123|name1|                    |null|sales|
| 456|name3|                    | 223|Legal|
|null|name2|                    | 456|   IT|
+----+-----+                    +----+-----+

>>> df1.join(df2, "id").show
+---+-----+-----+
| id| name| dept|
+---+-----+-----+
|123|name1|sales|
|456|name3|   IT|
+---+-----+-----+
```

Join types

SQL Join Type	In Spark (synonyms)	Description
INNER	`"inner"`	Data from left and right matching both ways (intersection)
FULL OUTER	`"outer"`, `"full"`, `"fullouter"`	All rows from left and right with extra data if present (union)
LEFT OUTER	`"leftouter"`, `"left"`	Rows from left with extra data from right if present
RIGHT OUTER	`"rightouter"`, `"right"`	Rows from right with extra data from left if present
LEFT SEMI	`"leftsemi"`	Data from left with a match with right
LEFT ANTI	`"leftanti"`	Data from left with no match with right
CROSS	`"cross"`	Cartesian product of left and right (never used)

Join types

Inner join ("inner")

```
>>> inner = df_left.join(df_right, "id", "inner")

df_left                 df_right
+---+-----+             +---+-----+
| id|value|             | id|value|
+---+-----+             +---+-----+
|  1|   A1|             |  3|   A3|
|  2|   A2|             |  4| A4_1|
|  3|   A3|             |  4|   A4|
|  4|   A4|             |  5|   A5|
+---+-----+             |  6|   A6|
                        +---+-----+
inner
+---+-----+-----+
| id|value|value|
+---+-----+-----+
|  3|   A3|   A3|
|  4|   A4|   A4|
|  4|   A4| A4_1|
+---+-----+-----+
```

Outer join ("outer", "full" or "fullouter")

```
>>> outer = df_left.join(df_right, "id", "outer")
df_left                 df_right
+---+-----+             +---+-----+
| id|value|             | id|value|
+---+-----+             +---+-----+
|  1|   A1|             |  3|   A3|
|  2|   A2|             |  4| A4_1|
|  3|   A3|             |  4|   A4|
|  4|   A4|             |  5|   A5|
+---+-----+             |  6|   A6|
                        +---+-----+
outer
+---+-----+-----+
| id|value|value|
+---+-----+-----+
|  1|   A1| null|
|  2|   A2| null|
|  3|   A3|   A3|
|  4|   A4|   A4|
|  4|   A4| A4_1|
|  5| null|   A5|
|  6| null|   A6|
+---+-----+-----+
```

Left join ("leftouter" or "left")

```
>>> left = df_left.join(df_right, "id", "left")

df_left                df_right
+---+-----+            +---+-----+
| id|value|            | id|value|
+---+-----+            +---+-----+
|  1|   A1|            |  3|   A3|
|  2|   A2|            |  4| A4_1|
|  3|   A3|            |  4|   A4|
|  4|   A4|            |  5|   A5|
+---+-----+            |  6|   A6|
                       +---+-----+
left
+---+-----+-----+
| id|value|value|
+---+-----+-----+
|  1|   A1| null|
|  2|   A2| null|
|  3|   A3|   A3|
|  4|   A4|   A4|
|  4|   A4| A4_1|
+---+-----+-----+
```

Right ("rightouter" or "right")

```
>>> right = df_left.join(df_right, "id", "right")

df_left                 df_right
+---+-----+             +---+-----+
| id|value|             | id|value|
+---+-----+             +---+-----+
|  1|   A1|             |  3|   A3|
|  2|   A2|             |  4| A4_1|
|  3|   A3|             |  4|   A4|
|  4|   A4|             |  5|   A5|
+---+-----+             |  6|   A6|
                        +---+-----+
right
+---+-----+-----+
| id|value|value|
+---+-----+-----+
|  3|   A3|   A3|
|  4|   A4|   A4|
|  4|   A4| A4_1|
|  5| null|   A5|
|  6| null|   A6|
+---+-----+-----+
```

Left semi join ("leftsemi")

```
>>> left_semi = df_left.join(df_right, "id", "leftsemi")

df_left                  df_right
+---+-----+              +---+-----+
| id|value|              | id|value|
+---+-----+              +---+-----+
|  1|   A1|              |  3|   A3|
|  2|   A2|              |  4| A4_1|
|  3|   A3|              |  4|   A4|
|  4|   A4|              |  5|   A5|
+---+-----+              |  6|   A6|
                         +---+-----+
left_semi
+---+-----+
| id|value|
+---+-----+
|  3|   A3|
|  4|   A4|
+---+-----+
```

Left anti joint ("leftanti")

```
>>> left_anti = df_left.join(df_right, "id", "leftanti")

df_left                 df_right
+---+-----+             +---+-----+
| id|value|             | id|value|
+---+-----+             +---+-----+
|  1|   A1|             |  3|   A3|
|  2|   A2|             |  4| A4_1|
|  3|   A3|             |  4|   A4|
|  4|   A4|             |  5|   A5|
+---+-----+             |  6|   A6|
                        +---+-----+
left_anti
+---+-----+
| id|value|
+---+-----+
|  1|   A1|
|  2|   A2|
+---+-----+
```

3.7. Aggregations

Performing aggregations

- Maybe **the most used operations** in `SQL` and `Spark SQL`
- Similar to `SQL`, we use `"group by"` to perform **aggregations**
- We usually can call the aggregation function just after `groupBy` Namely, we use `groupBy().agg()`
- **Many aggregation functions** in `pyspark.sql.functions`
- Some examples:
  - Numerical: `fn.avg`, `fn.sum`, `fn.min`, `fn.max`, etc.
  - General: `fn.first`, `fn.last`, `fn.count`, `fn.countDistinct`, etc.

Examples

```
>>> from pyspark.sql import functions as fn

>>> products = spark.createDataFrame([
        ('1', 'mouse', 'microsoft', 39.99),
        ('2', 'mouse', 'microsoft', 59.99),
        ('3', 'keyboard', 'microsoft', 59.99),
        ('4', 'keyboard', 'logitech', 59.99),
        ('5', 'mouse', 'logitech', 29.99),
    ], ['prod_id', 'prod_cat', 'prod_brand', 'prod_value'])

>>> products.groupBy('prod_cat').avg('prod_value').show()

>>> # Or
>>> products.groupBy('prod_cat').agg(fn.avg('prod_value')).show()
+--------+-----------------+
|prod_cat|  avg(prod_value)|
+--------+-----------------+
|keyboard|            54.99|
|   mouse|43.32333333333333|
+--------+-----------------+
```

Examples

```
>>> from pyspark.sql import functions as fn

>>> products.groupBy('prod_brand', 'prod_cat')\
        .agg(fn.avg('prod_value')).show()
+----------+--------+---------------+
|prod_brand|prod_cat|avg(prod_value)|
+----------+--------+---------------+
| microsoft|   mouse|          49.99|
|  logitech|keyboard|          49.99|
| microsoft|keyboard|          59.99|
|  logitech|   mouse|          29.99|
+----------+--------+---------------+
```

Examples

```
>>> from pyspark.sql import functions as fn

>>> products.groupBy('prod_brand').agg(
        fn.round(fn.avg('prod_value'), 1).alias('average'),
        fn.ceil(fn.sum('prod_value')).alias('sum'),
        fn.min('prod_value').alias('min')
    ).show()
+----------+-------+---+-----+
|prod_brand|average|sum|  min|
+----------+-------+---+-----+
|  logitech|   40.0| 80|29.99|
| microsoft|   53.3|160|39.99|
+----------+-------+---+-----+
```

Examples

```
>>> # Using an SQL query
>>> products.createOrReplaceTempView("products")

>>> query = """
    SELECT
    prod_brand,
    round(avg(prod_value), 1) AS average,
    min(prod_value) AS min
    FROM products
    GROUP BY prod_brand
    """

>>> spark.sql(query).show()
+----------+-------+-----+
|prod_brand|average|  min|
+----------+-------+-----+
|  logitech|   40.0|29.99|
| microsoft|   53.3|39.99|
+----------+-------+-----+
```

3.8. Window functions

Window functions

- A very, very **powerful feature**
- They allow to solve **complex problems**
- They exist in some relational databases
- A good article about this feature is `[here]`

Window functions

- It's **similar to aggregations**, but the **number of rows doesn't change**
- Instead, **new columns are created**, and the **aggregated values are duplicated** for values of the same "group"
- There are "tradictional" aggregations, such as `min`, `max`, `avg`, `sum`
- and "special" types, such as `lag`, `lead`, `rank`

Numerical window functions

```
>>> from pyspark.sql import Window
>>> from pyspark.sql import functions as fn

>>> # First, we create the Window definition
>>> window = Window.partitionBy('prod_brand')

>>> # Then, we can use "over" to aggregate on this window
>>> avg = fn.avg('prod_value').over(window)

>>> # Finally, we can it as a classical column
>>> products.withColumn('avg_brand_value', fn.round(avg, 2)).show()
+-------+--------+----------+----------+---------------+
|prod_id|prod_cat|prod_brand|prod_value|avg_brand_value|
+-------+--------+----------+----------+---------------+
|      4|keyboard|  logitech|     49.99|          39.99|
|      5|   mouse|  logitech|     29.99|          39.99|
|      1|   mouse| microsoft|     39.99|          53.32|
|      2|   mouse| microsoft|     59.99|          53.32|
|      3|keyboard| microsoft|     59.99|          53.32|
+-------+--------+----------+----------+---------------+
```

Numerical window functions

```
>>> from pyspark.sql import Window
>>> from pyspark.sql import functions as fn

>>> # The window can be defined on multiple columns
>>> window = Window.partitionBy('prod_brand', 'prod_cat')

>>> avg = fn.avg('prod_value').over(window)

>>> products.withColumn('avg_value', fn.round(avg, 2)).show()
+-------+--------+----------+----------+---------+
|prod_id|prod_cat|prod_brand|prod_value|avg_value|
+-------+--------+----------+----------+---------+
|      1|   mouse| microsoft|     39.99|    49.99|
|      2|   mouse| microsoft|     59.99|    49.99|
|      4|keyboard|  logitech|     49.99|    49.99|
|      3|keyboard| microsoft|     59.99|    59.99|
|      5|   mouse|  logitech|     29.99|    29.99|
+-------+--------+----------+----------+---------+
```

Numerical window functions

```
>>> from pyspark.sql import Window
>>> from pyspark.sql import functions as fn

>>> # Multiple windows can be defined
>>> window1 = Window.partitionBy('prod_brand')
>>> window2 = Window.partitionBy('prod_cat')

>>> avg_brand = fn.avg('prod_value').over(window1)
>>> avg_cat = fn.avg('prod_value').over(window2)

>>> products \
        .withColumn('avg_by_brand', fn.round(avg_brand, 2)) \
        .withColumn('avg_by_cat', fn.round(avg_cat, 2)) \
        .show()
+-------+--------+----------+----------+------------+----------+
|prod_id|prod_cat|prod_brand|prod_value|avg_by_brand|avg_by_cat|
+-------+--------+----------+----------+------------+----------+
|      4|keyboard|  logitech|     49.99|       39.99|     54.99|
|      3|keyboard| microsoft|     59.99|       53.32|     54.99|
|      5|   mouse|  logitech|     29.99|       39.99|     43.32|
|      1|   mouse| microsoft|     39.99|       53.32|     43.32|
|      2|   mouse| microsoft|     59.99|       53.32|     43.32|
+-------+--------+----------+----------+------------+----------+
```

Lag and Lead

- `lag` and `lead` are special functions used over an **ordered window**

- They are used to **take the "previous" and "next" value** within the window

- Very useful in datasets with a date column for instance

Lag and Lead

```
>>> purchases = spark.createDataFrame([
        (date(2017, 11, 1), 'mouse'),
        (date(2017, 11, 2), 'mouse'),
        (date(2017, 11, 4), 'keyboard'),
        (date(2017, 11, 6), 'keyboard'),
        (date(2017, 11, 9), 'keyboard'),
        (date(2017, 11, 12), 'mouse'),
        (date(2017, 11, 18), 'keyboard')
    ], ['date', 'prod_cat'])
>>> purchases.show()
+----------+--------+
|      date|prod_cat|
+----------+--------+
|2017-11-01|   mouse|
|2017-11-02|   mouse|
|2017-11-04|keyboard|
|2017-11-06|keyboard|
|2017-11-09|keyboard|
|2017-11-12|   mouse|
|2017-11-18|keyboard|
+----------+--------+
```

Lag and Lead

```
>>> window = Window.partitionBy('prod_cat').orderBy('date')

>>> prev_purch = fn.lag('date', 1).over(window)
>>> next_purch = fn.lead('date', 1).over(window)

>>> purchases\
        .withColumn('prev', prev_purch)\
        .withColumn('next', next_purch)\
        .orderBy('prod_cat', 'date')\
        .show()
+--------+----------+----------+----------+
|prod_cat|      date|      prev|      next|
+--------+----------+----------+----------+
|keyboard|2017-11-04|      null|2017-11-06|
|keyboard|2017-11-06|2017-11-04|2017-11-09|
|keyboard|2017-11-09|2017-11-06|2017-11-18|
|keyboard|2017-11-18|2017-11-09|      null|
|   mouse|2017-11-01|      null|2017-11-02|
|   mouse|2017-11-02|2017-11-01|2017-11-12|
|   mouse|2017-11-12|2017-11-02|      null|
+--------+----------+----------+----------+
```

Rank, DenseRank and RowNumber

- Another set of **useful "special" functions**
- Also used on **ordered windows**
- They create a **rank**, or an **order** of the items within the window

Rank and RowNumber

```
>>> contestants = spark.createDataFrame([
        ('veterans', 'John', 3000),
        ('veterans', 'Bob', 3200),
        ('veterans', 'Mary', 4000),
        ('young', 'Jane', 4000),
        ('young', 'April', 3100),
        ('young', 'Alice', 3700),
        ('young', 'Micheal', 4000),
    ], ['category', 'name', 'points'])

>>> contestants.show()
+--------+-------+------+
|category|   name|points|
+--------+-------+------+
|veterans|   John|  3000|
|veterans|    Bob|  3200|
|veterans|   Mary|  4000|
|   young|   Jane|  4000|
|   young|  April|  3100|
|   young|  Alice|  3700|
|   young|Micheal|  4000|
+--------+-------+------+
```

Rank and RowNumber

```
>>> window = Window.partitionBy('category')\
        .orderBy(contestants.points.desc())

>>> rank = fn.rank().over(window)
>>> dense_rank = fn.dense_rank().over(window)
>>> row_number = fn.row_number().over(window)

>>> contestants\
        .withColumn('rank', rank)\
        .withColumn('dense_rank', dense_rank)\
        .withColumn('row_number', row_number)\
        .orderBy('category', fn.col('points').desc())\
        .show()
+--------+-------+------+----+----------+----------+
|category|   name|points|rank|dense_rank|row_number|
+--------+-------+------+----+----------+----------+
|veterans|   Mary|  4000|   1|         1|         1|
|veterans|    Bob|  3200|   2|         2|         2|
|veterans|   John|  3000|   3|         3|         3|
|   young|   Jane|  4000|   1|         1|         1|
|   young|Micheal|  4000|   1|         1|         2|
|   young|  Alice|  3700|   3|         2|         3|
|   young|  April|  3100|   4|         3|         4|
+--------+-------+------+----+----------+----------+
```

3.9. Writing dataframes

Writing dataframes

- Very **similar to reading**. Output formats are the same: `csv`, `json`, `parquet`, `orc`, `jdbc`, etc. Note that `write` **is an action**
- Instead of `df.read.{source}` use `df.write.{target}`
- Main option is `mode` with possible values:
  - `"append"`: append contents of this `DataFrame` to existing data.
  - `"overwrite"`: overwrite existing data
  - `"error"`: throw an exception if data already exists
  - `"ignore"`: silently ignore this operation if data already exists.

Example

```
>>> products.write.csv('/products.csv')
>>> products.write.mode('overwrite').parquet('/file.parquet')
>>> products.write.format('parquet').save('/file.parquet')
```

3.10. Under the hood...

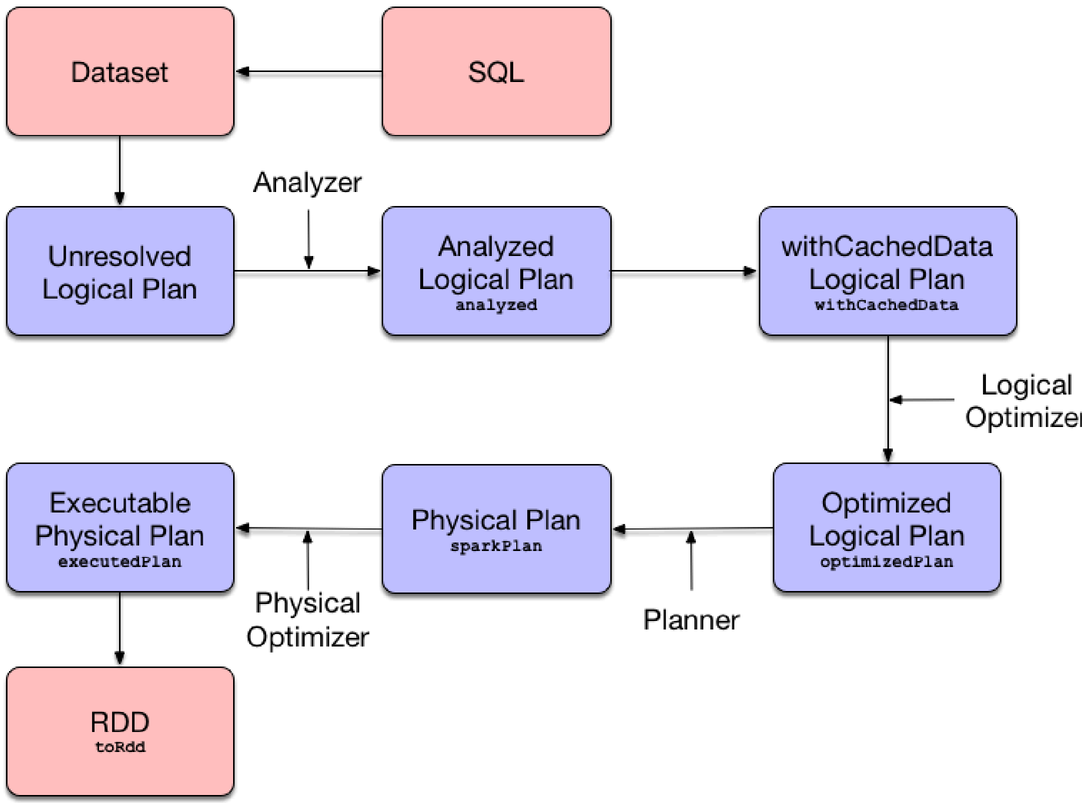
Query planning and optimization

A **lot happens under the hood** when executing an **action** on a `DataFrame`.
The query goes through the following **exectution stages**:

1. Logical Analysis
2. Caching Replacement
3. Logical Query Optimization (using rule-based and cost-based optimizations)
4. Physical Planning
5. Physical Optimization (e.g. Whole-Stage Java Code Generation or Adaptive Query Execution)
6. Constructing the RDD of Internal Binary Rows (that represents the structured query in terms of Spark Core's RDD API)

`https://jaceklaskowski.gitbooks.io/mastering-spark-sql/spark-sql.html`

Query planning and optimization



https://jaceklaskowski.gitbooks.io/mastering-spark-sql/spark-sql.html